

7. Threading

Last edited time	October 14, 2026 8:56 PM
Practica	☐
Teoria	☑

Threading

Introducción

- El concepto de **thread** nos permite definir múltiples flujos de instrucciones dentro del mismo código en nuestro programa.

▼ Con respecto a procesos y núcleos

▼ **Proceso**

> ***Programa en ejecución*** *que ocupa espacio de la memoria RAM y ejecuta un flujo de instrucciones*

- En general, cada **núcleo** de nuestra CPU ejecuta el flujo de instrucciones de un proceso distinto.
  - Si hay más procesos que núcleos, se forma una cola de procesos que se van activando y suspendiendo regularmente, para que todos puedan ejecutarse.
- Podemos hacer que **varios núcleos** ejecuten los diferentes flujos de instrucciones de un **mismo proceso** al mismo tiempo.

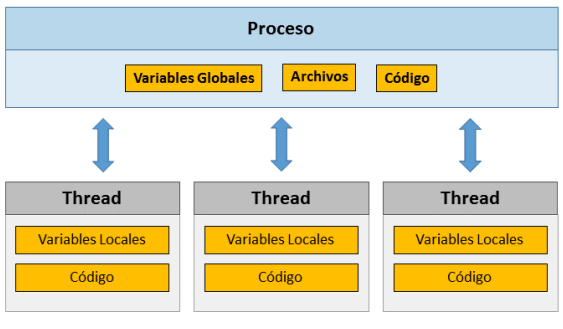
▼ Paralelismo vs concurrencia

- **Paralelismo** → ejecución simultánea de varias tareas en **diferentes núcleos** o procesadores.
  - No es posible en Python por el **GIL (*Global Interpreter Lock*)**
    - Independiente de la cantidad de núcleos de la CPU, evita ejecutar threads en paralelo.
    - Para lograr paralelismo para operaciones intensivas de CPU se usan librerías como `multiprocessing` o se desactiva manualmente el **GIL.**
- **Concurrencia** → gestionar varias tareas de forma intercalada en un **mismo núcleo** o procesador.
  - Esto es realizable en Python a través del módulo `threading`.

Threads

- **Thread** (o hilo): unidad de ejecución de código dentro de un proceso.
- Todo proceso se crea con al menos un thread (**thread principal**). Además, se pueden crear más threads para que el programa ejecute varias partes de código simultáneamente.

▼ Cada thread lleva su propio registro de variables locales y en qué parte del programa se encuentra.



▼ Los threads se ejecutan de manera concurrente a través de **thread scheduling** o **time slicing**

1. Se escoge un thread entre los disponibles para ejecutar.
2. Se ejecuta ese thread durante un cierto tiempo.
3. Se deja el thread actual en espera.
4. Se vuelve al paso 1.

👁️ Usar threads en una CPU con un sólo núcleo hace que el programa se ejecute **más lento**, puesto que hay que ocupar tiempo en controlar los turnos de los threads.
Sin embargo, la programación es mucho más clara y simple a través del uso de threads.

- **Usos de threads**

  ▼ Separar funciones independientes que trabajan sobre un mismo conjunto de datos pero pueden ejecutarse simultáneamente

  ▼ Ejemplo de **modelo productor-consumidor**

  - Un thread que recibe input y lo agrega a una cola de texto para procesar.
  - Otro thread que procesa línea a línea y los saca de la cola de procesamiento.

  ▼ Interfaces gráficas

  - Necesita interactuar con el usuario mientras se ejecuta algún thread de cómputo pesado.

  ▼ Aplicaciones multiusuario

  - Cada thread se encargaría de las peticiones de cada usuario independientemente.

`threading`

- La clase `Thread` del módulo `threading` representa un hilo (o thread).

▼ Las instrucciones que ejecuta cada thread se pasan a través del parámetro `target` **, el cual recibe el nombre de una sola función.**

```
from threading import Thread

def funcion1():
    print("hola")

def funcion2():
    funcion1()
    print("adios")

hilo_fx_simple = Thread(target=funcion1)
hilo_fx_contenedora = Thread(target=funcion2)
```

▼ La clase `Thread` admite el argumento `name` , para asignarle un nombre a cada thread.

▼ Ejemplo

```python
def saludar() -> None:
    thread_actual = threading.current_thread()
    print(f"Hola desde {thread_actual.name}\n")


hilo_1 = threading.Thread(name="Mi thread 1", target=saludar)
hilo_2 = threading.Thread(name="Mi thread 2", target=saludar)
```

Para acceder al nombre de un thread se ocupa `thread.name`

- En el caso del thread principal, su nombre es `MainThread`
- Si no se especifica, se crea un nombre único de la forma “Thread-N” o “Thread-N (target)” (donde target es `target.__name__` si se le pasó un `target`)

▼ Se le pueden pasar los argumentos que recibe el `target` a través de los parámetros `args` y `kwargs`

- `args` es una lista o una tupla de argumentos
- `kwargs` es un diccionario de argumentos por palabra clave.

- Para ejecutar un thread se ocupa el método `start()`
  - No es posible ejecutar más de una vez el mismo thread.
- La función `current_thread()` del módulo `threading` retorna una referencia de la instancia del thread que está ejecutando actualmente el código.

👁️ `time.sleep(n)` detiene la ejecución del thread que lo llama por `n` segundos.

Threads personalizados

- En lugar de crear funciones para pasarle las instrucciones a los threads, se puede crear directamente subclases de `Thread` que ejecute dichas instrucciones cuando sean ejecutadas. Solo hay dos consideraciones:
  - El método `__init__` de la subclase debe comenzar llamando al inicializador de `Thread`
    - `super().__init__(name=nombre)`
  - Debemos hacer override del método `run` , que es el que ejecutamos con el método `start()`

▼ Ejemplo

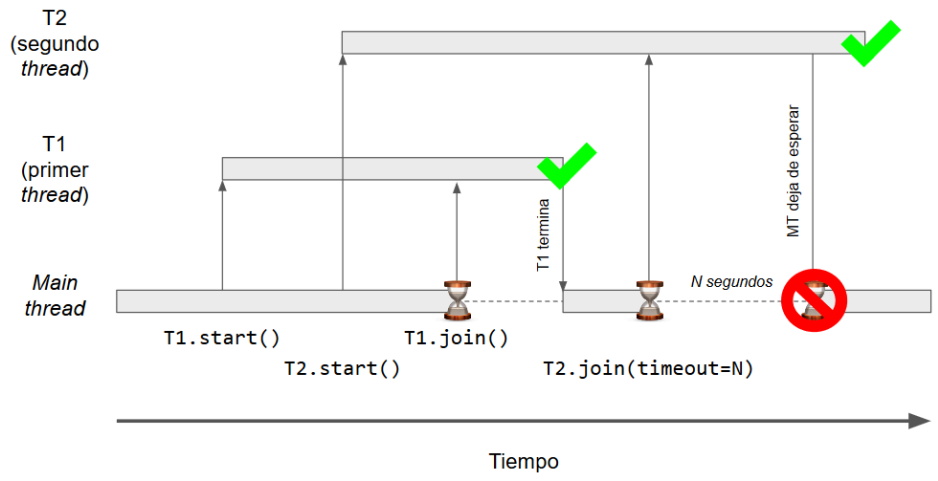
```python
import threading


class CuentaOvejas(threading.Thread): # Hereda de Thread
    """Este será nuestro nuevo Cuenta Ovejas basado en Thread"""

    def __init__(self, nombre: str, max_ovejas: int) -> None:
        # En el caso de los threads, lo primero es invocar al init original. SIEMPR
E.
        super().__init__(name=nombre)
        self.max_ovejas = max_ovejas # Se agrega un atributo de instancia extra

    def run(self) -> None:
        # Este metodo define las instrucciones a ejecutar de este thread
        # cuando ejecutamos el metodo start()
```

Esperando otros threads



- Para esperar que se termine de ejecutar un thread (o un grupo de estos) se ocupa `thread_objetivo.` **`join()`** donde `thread_objetivo` es el thread que queremos esperar desde el thread donde se ejecutó esta línea.
  - Se le puede especificar **`timeout`** `=n` para que solo se espere `n` segundos y luego continuar.

`is_alive()`

- Este método retorna `True` si el thread todavía está en funcionamiento.
- Usualmente se ocupa después de `join(timeout=n)` para saber si se alcanzó a ejecutar por completo.

Daemons

- Los **daemon threads** son threads que, a pesar de que aún estén corriendo, no impiden que el programa principal termine.
- Se puede crear de 2 formas
  1. Instanciando una clase `Thread` y

     ▼ A) Especificar el parámetro `daemon` con el resto de argumentos

     `dormilon = threading.Thread(name="Dormilón", target=dormilon, daemon=True )`

     - B) Actualizándolo después de creado
       - Esto sólo es posible si aún no se ejecuta el thread.

     ▼ Ejemplo

     ```
     con_insomnio = threading.Thread(name="Con insomnio", target=con_insomnio)
     con_insomnio.daemon = True
     ```

  2. Creando una subclase de `Thread` que se instancie con el atributo en `True`

     ▼ Ejemplo

     ```
     class Daemon(threading.Thread):

         def __init__(self) -> None:
             super().__init__()
             # Cuando inicializamos el thread lo declaramos como daemon
             self.daemon = True

         def run(self):
             print("Daemon thread: Empezando...")
             time.sleep(2)
             print("Daemon thread: Terminando...")
     ```

- Es posible esperar a que termine un daemon thread usando el método `join()`

`Timer`

- Subclase de `Thread` que, una vez ejecutado, permite realizar una acción después que ha pasado un determinado tiempo.
- Sintaxis

  `Timer(interval, function, args=None, kwargs=None)`

  - `interval` es un período de tiempo en segundos
  - `args` acepta una tupla/lista y `kwargs` un diccionario
- Después de hacer uso de `start()` , es posible detenerlo usando el método `cancel()`
  - Esto solo se puede hacer si aún no se cumple el tiempo que tiene que esperar para ejecutar la acción.

▼ Es posible implementar nuestra propia subclase que sea equivalente a un `Timer`

```
import time


class MiTimer(threading.Thread):
    def __init__(self, tiempo_espera: int) -> None:
        super().__init__()
        self.tiempo_espera = tiempo_espera
        self._cancel = False

    def cancel(self) -> None:
        self._cancel = True

    def run(self) -> None:
        time.sleep(self.tiempo_espera)
        if self._cancel == True:
            return
        # implementar acción:
```

Concurrencias y secuencias

▼ Operación atómica

- Operación que es ejecutable solo por un thread en todo momento.

▼ Sección crítica

- Secuencia de instrucciones que no queremos que sean ejecutadas en paralelo.

Mecanismos de sincronización

- Permiten controlar el acceso a un recurso cuando existe concurrencia múltiple.

`Lock`

- El `lock`es una primitiva de sincronización de threads, provista por la clase `Lock` de `threading` .
  - Se utiliza para que sólo un thread pueda estar en una misma sección crítica a la vez.
- Un `lock` puede estar bloqueado o desbloqueado (inicialmente está desbloqueado).

▼ Mecanismo de funcionamiento

1. Si un thread quiere entrar a una sección crítica, primero debe adquirir el `lock` mediante la operación `acquire()`
2. Cuando lo consigue, este queda bloqueado, haciendo que deban esperar los otros threads que quieran adquirir el mismo `lock`

3. Cuando el thread quiera salir de la sección crítica, debe liberar el `lock` mediante `release()` , lo cual permite que otros threads lo puedan adquirir.

👁️ El `lock` de una sección crítica **debe ser el mismo** para todos los threads que accedan a dicha sección.

📌

▼ Los `lock` se pueden manejar dentro de un **context manager** con la sentencia `with` .
→ `with` se va a encargar de llamar los métodos `acquire()` y `release()`

```python
def sumador(contador: int, lock: threading.Lock) -> None:
    nombre = threading.current_thread().name
    for _ in range(10):
        with lock:
            # --- Sección crítica ---.
            # Está garantizado que en estas líneas sólo habrá un thread a la
vez.
            valor = contador.valor
            print(f"{nombre}: lee {valor}")
            nuevo_valor = valor + 1
            print(f"{nombre}: suma 1 => {nuevo_valor}")
            contador.valor = nuevo_valor
            print(f"{nombre}: guarda {nuevo_valor}")
            time.sleep(1)
            # --- Fin de la sección crítica ---.
```

Event

- Clase de la librería `threading`
- Permite que los threads se comuniquen entre sí.
- En lugar de esperar que un thread acabe por completo (usando `join()`), se puede esperar a que mande un “mensaje” (un evento).
- Un objeto `Event` tiene un **flag** interno que puede ser `True` (señal activa) o `False` (señal inactiva).
- Métodos de eventos

  ▼ `wait()` → detiene el thread que lo ejecuta, hasta que el otro thread active la señal.

  - (en caso de que ya esté activa, sigue funcionando altiro)
  - `set()` → activa la señal vs `clear()` → resetea la señal (la deja inactiva)
  - `is_set()` → retorna `True` o `False` indicando el valor del flag, sin pausar el thread que lo ejecuta.

▼ Ejemplo de uso

```python
# Ejemplo adaptado de http://zulko.github.io/blog/2013/09/19/a-basic-example-of-thr
eads-synchronization-in-python/

import threading
import time


# Tenemos dos eventos o señales.
# Esta es para avisar que el video ya está listo para ser reproducido.
video_cargado = threading.Event()

# Esta es para avisar que el audio ya está listo para ser reproducido.
audio_cargado = threading.Event()
```

```python
def hora_actual() -> None:
    return time.ctime().split(" ")[3]


def reproducir_video(nombre: str) -> None:
    print(f"[{hora_actual()}] Cargando video {nombre}. Se demorará 10 segundos\n")
    # Supongamos que se demora 10 segundos
    time.sleep(10)
    print(f"[{hora_actual()}] Video cargado. Esperando audio")
    # Avisamos que el video ya está cargado
    video_cargado.set()
    # Esperamos a que el audio ya se haya cargado
    audio_cargado.wait()
    # ¡Listo!
    print(f"[{hora_actual()}] Comenzando reproducción del video")


def reproducir_audio(nombre: str) -> None:
    print(f"[{hora_actual()}] Cargando audio {nombre}. Se demorará 3 segundos\n")
    # Supongamos que se demora 3 segundos
    time.sleep(3)
    print(f"[{hora_actual()}] Audio cargado. Esperando video")
    # Avisamos que el audio ya está cargado
    audio_cargado.set()
    # Esperamos a que el video ya se haya cargado
    video_cargado.wait()
    # ¡Listo!
    print(f"[{hora_actual()}] Comenzando reproducción del audio")


t1 = threading.Thread(target=reproducir_audio, args=("Your Name - Zen Zen Zense",))
t2 = threading.Thread(target=reproducir_video, args=("Your Name - Zen Zen Zense",))

t1.start()
t2.start()

t1.join()
t2.join()
```

👁️ Python considera el número `0` como `False` y todo el resto de enteros como `True` .
Esto implica que `if not n%4:` equivale a `if n%4 == 0:`

Deadlocks

- Situaciones donde dos o más threads se esperan mutuamente, sin que ninguno finalmente avance.
- Sinónimos: interbloqueo, **abrazo mortal,** etc.

Ejemplos y aplicaciones

👁️ Cuando uno ejecuta `self.atributo` , primero se revisa en la instancia y después a nivel de clase. En caso de no encontrarse en ninguna, se retorna `AttributeError`

- `lock` como atributo de clase en una subclase de `Thread`
  - Permite generar instancias que compartan el mismo lock desde su creación.

- El **patrón productor-consumidor** se origina cuando dos o más threads acceden a un mismo espacio de almacenamiento o ***buffer.***
  - Los productores ponen ítems en el buffer y los consumidores sacan elementos del buffer.
  - En general se implementa con una **cola sincronizada (o thread-safe)** que funciona correctamente al ser usada por múltiples threads.

> 👁️ Una **cola sincronizada** podría implementarse con una `deque` , pero igual habría que manejar `lock` , `Event` y otros para que funcione correctamente. En su lugar, existe el módulo `queue` que ya tiene toda esa logística implementada.

`queue`

- Implementa colas para multi-productores y multi-consumidores, es decir, **colas thread-safe.**

▼ Clases

- `Queue(maxsize=0)`
  - Cola FIFO
- `LifoQueue(maxsize=0)`
- `PriorityQueue(maxsize=0)`
  - Los ítems de menor valor son obtenidos primero.
- `SimpleQueue`
  - Cola FIFO sin límites.
  - Carecen de funcionalidades avanzadas como rastreo de tareas.

▼ Excepciones

- `Empty` → ocurre cuando se ejecuta un `get(block=False)` o `get_nowait()` sobre un objeto `Queue`
- `Full` → ocurre cuando se ejecuta un `put(block=False)` o `put_nowait()` sobre un objeto `Queue`
- `Shutdown` → ocurre cuando se ejecuta `put()` o `get()` sobre un objeto `Queue` que ya ha sido desactivado

▼ Métodos de objetos `Queue`

- `qsize()` → retorna el tamaño aproximado de la cola.
- `empty()` → retorna `True` si la cola está vacía.
- `full()` → retorna `True` si la cola está llena.

▼ **`put`** `(item, block=True, timeout=None)`

- Pone un ítem en la cola
- Si `block` es `True` , el thread que lo ejecuta se va a bloquear hasta que haya un espacio vacío en la cola.
- Si se especifica los segundos de `timeout` , entonces va a esperar ese tiempo antes de poner `item` en la cola.
- En caso contrario, inmediatamente va a tratar de poner `item` en la cola.

- `put_nowait(item)`
  - Equivale a `put(item, block=False)`
- **`get`** `(block=True, timeout=None)`
  - Remueve y retorna un ítem de la cola.
- `get_nowait()`
  - Equivale a `get(False)`

▼ `task_done()`

- Indica que una tarea que estaba en la cola ya fue completada.
- Por cada `get()` usado para obtener una tarea, se debe llamar después a `task_done()` cuando esta ya haya sido procesada.
- Levanta un `ValueError` si se llama más veces que ítems había en la cola.

▼ `join()`

- Bloquea el thread que lo ejecuta hasta que todos los ítems en la cola han sido obtenidos y procesados.
- Con cada ítem que se agrega a la cola, sube el contador de tareas por hacer.